

南京信息工程大学 期中项目报告

课程名称 算法分析与设计 教师 待补充

专业 待补充 班级 待补充

学生姓名 李林浩

学生学号 202483290054

任务描述和要求：

算法课期中项目为单人游戏中的算法策略设计任务，学生以小组为单位，访问 OJ 系统并查看题目：“Project Block Sequence Grand Master! feat. Jiahao”，在题目中所给的辅助代码基础上，修改策略函数优化算法。每个可以正常运行的策略都会得到一个分数，没有绝对意义上的正确和错误，应该力争获取更高的分数。

请按照要求完成以下报告：

数据使用情况	本策略同时使用了题目给出的静态棋盘信息和可预测的未来掉落信息。 - 当前棋盘 <code>grid</code>：读取每个格子的颜色、是否为炸弹、是否为 <code>wildcard</code>，用于构建连通块与可行路径。 - 关卡信息 <code>level</code>、棋盘边长 <code>N</code>、步数 <code>step</code>：决定不同关卡和不同阶段采用不同的搜索参数。 - 随机种子 <code>seed</code> 生成的 <code>drop_queue</code> 与 <code>queue_ptr</code>：在本地沙盘中精确预览执行某条路径后的新棋盘。 - 连通块统计信息：包括组件大小、节点度数、边缘深度、剩余出口数和可扩展分支数。 - 路径评价信息：当前路径得分、未来搜索得分、纯色局面结构分、<code>wildcard</code> 桥点消耗等。
算法策略	最终代码采用“分关卡 + 分阶段 + 多步预览”的策略框架。 - 共性框架：先按颜色与 <code>wildcard</code> 构造连通块，再在每个连通块内做 DFS 路径搜索，保留高质量候选，最后进行 3 层左右的未来评估。 - 第 1/2 关（纯色关）：目标不是只吃眼前高分，而是优先制造更大的后续连通块。针对较大的组件，起点更偏向中心和高连接度节点，避免过早修边角；评价函数同时考虑当前分数、链长偏好和局部未来潜力。 - 第 3 关（<code>wildcard</code> 关）：在保持一定贪心性的同时，增加 <code>wildcard</code> 桥点保护，尽量避免过早消耗连接度高的 <code>wildcard</code>，从而维持中后期大链能力。

	4. 第 4/5 关（炸弹关）：保留更稳健的未来权重，优先使用沙盘预览判断炸弹爆炸收益与后续死局风险，避免因局部贪心破坏整体布局。 5. 针对性修复：本次最终代码把“结构潜力”前移到候选池筛选阶段，使真正有后续价值的铺垫招法能进入最终比较，而不是在早期就被短链高分候选挤掉。
迭代过程	本项目不是一次成型，而是围绕 OJ 实测结果持续迭代。 1. high_ceiling：总分 23952。特点是后两关较稳，但前两关偏保守。 2. aggressive：总分 23632。前两关明显提升到 3194/3194，但第 4/5 关被过度激进的候选排序拖累。 3. greedy_hybrid：总分 22864。第 3 关 wildcard 关提升到 4778，但炸弹关损失较大。 4. specialized：总分 24312。开始按关卡拆分策略，后两关恢复稳定，第 1/2 关也超过早期版本。 5. level_phased：总分 24676（已完整验证的最新成绩）。进一步把第 1/2/3 关按前中后期分段处理，第 1/2 关分别达到 3298 和 3222。 6. targeted_fix（本报告附录代码）：基于手工复盘继续修复三个缺陷：纯色关中盘爱吃小块、结构分介入过晚、wildcard 桥点消耗过早。该版本作为下一轮冲分代码保存。
AI 工具使用情况	本项目使用了 AI 编程助手 Codex 进行分析与辅助实现。 1. 读取并比较多次 OJ 原始导出结果，自动提取分关卡成绩和路径长度特征。 2. 对不同版本策略进行代码对比，定位候选筛选、路径扩展顺序、关卡参数设置等差异。 3. 根据手工复盘结论生成针对性修复版代码，并协助整理为实验报告初稿。 4. 人工负责确定优化方向、判断哪些修改真正符合题目特点，并对最终文档内容进行确认。
源码	完整源码见附录 A。 附录源码文件名：block-sequence-grand-master_solver_targeted_fix.cpp 说明：报告中的策略描述与附录代码一致；已完整验证的最高分来自同一策略框架下的上一轮稳定版本。
成绩	当前已完整验证的最高总分：24676 分关卡成绩：第 1 关 3298，第 2 关 3222，第 3 关 4440，第 4 关 5508，第 5 关 8208

截图

下图根据 OJ 原始导出结果整理，用于报告归档。

OJ Highest Verified Result

Match ID: 23577

Game: Project Block Sequence Grand Master! feat. Jiahao

FINAL_SCORE = 24676

LEVEL_01 = 3298

LEVEL_02 = 3222

LEVEL_03 = 4440

LEVEL_04 = 5508

LEVEL_05 = 8208

Code appendix in report: targeted_fix version

Verified score source: latest exported OJ result

附录 A 最终策略源码 (targeted_fix 版本)

说明：以下代码为本次手工复盘后形成的针对性修复版。为便于查阅，附录保留完整源码，并在每行前增加行号。

```
0001 | #if __cplusplus < 201700L
0002 | #error "C++17 required"
0003 | #endif
0004 |
0005 | #include <algorithm>
0006 | #include <array>
0007 | #include <cmath>
0008 | #include <cstdint>
0009 | #include <iostream>
0010 | #include <limits>
0011 | #include <queue>
0012 | #include <random>
0013 | #include <sstream>
0014 | #include <string>
0015 | #include <unordered_set>
0016 | #include <vector>
0017 |
0018 | // =====
0019 | // 数据模型（带底层完美沙盘推演）
0020 | // =====
0021 |
0022 | struct Cell {
0023 |     int value = 1;
0024 |     int color() const { return std::abs(value); }
0025 |     bool is_bomb() const { return value < 0; }
0026 |     bool is_wildcard() const { return value == 0; }
0027 | };
0028 |
0029 | struct Board {
0030 |     int N = 0;
0031 |     int level = 1;
0032 |     std::vector<std::vector<Cell>> grid;
0033 |
0034 |     // 底层引擎变量：未来掉落物预知队列
0035 |     std::vector<std::vector<int>> drop_queue;
0036 |     std::vector<int> queue_ptr;
0037 |
0038 |     explicit Board(int n = 0) : N(n), grid(n, std::vector<Cell>(n)) {}
0039 | }
```

```

0040 |     Cell&      at(int r, int c)      { return grid[r][c]; }
0041 |     const Cell& at(int r, int c) const { return grid[r][c]; }
0042 |     bool in_bounds(int r, int c) const { return r >= 0 && r < N && c >= 0 && c < N; }
0043 |
0044 |     Board preview(const std::vector<std::pair<int,int>>& path) const {
0045 |         Board next_b = *this;
0046 |         if (path.size() < 2) return next_b;
0047 |
0048 |         std::vector<std::vector<bool>> in_path(N, std::vector<bool>(N, false));
0049 |         for (auto [r, c] : path) in_path[r][c] = true;
0050 |
0051 |         std::vector<std::vector<bool>> to_remove = in_path;
0052 |
0053 |         if (level >= 4) {
0054 |             for (auto [r, c] : path) {
0055 |                 if (!at(r, c).is_bomb()) continue;
0056 |                 for (int dr = -1; dr <= 1; ++dr) {
0057 |                     for (int dc = -1; dc <= 1; ++dc) {
0058 |                         int nr = r + dr, nc = c + dc;
0059 |                         if (in_bounds(nr, nc) && !in_path[nr][nc]) {
0060 |                             to_remove[nr][nc] = true;
0061 |                         }
0062 |                     }
0063 |                 }
0064 |             }
0065 |         }
0066 |
0067 |         for (int c = 0; c < N; ++c) {
0068 |             std::vector<Cell> remaining;
0069 |             for (int r = 0; r < N; ++r) {
0070 |                 if (!to_remove[r][c]) remaining.push_back(at(r, c));
0071 |             }
0072 |             int empty = N - static_cast<int>(remaining.size());
0073 |             for (int i = 0; i < empty; ++i) {
0074 |                 int val = next_b.drop_queue[c][next_b.queue_ptr[c]++];
0075 |                 next_b.at(i, c).value = val;
0076 |             }
0077 |             for (int i = 0; i < static_cast<int>(remaining.size()); ++i) {
0078 |                 next_b.at(empty + i, c) = remaining[i];
0079 |             }
0080 |         }
0081 |         return next_b;
0082 |     }
0083 |

```

```

0084 |     bool is_deadlocked() const {
0085 |         for (int r = 0; r < N; ++r) {
0086 |             for (int c = 0; c < N; ++c) {
0087 |                 int ac = at(r, c).color();
0088 |                 if (c + 1 < N) {
0089 |                     int ac2 = at(r, c + 1).color();
0090 |                     if (ac == ac2 || ac == 0 || ac2 == 0) return false;
0091 |                 }
0092 |                 if (r + 1 < N) {
0093 |                     int ac2 = at(r + 1, c).color();
0094 |                     if (ac == ac2 || ac == 0 || ac2 == 0) return false;
0095 |                 }
0096 |             }
0097 |         }
0098 |         return true;
0099 |     }
0100 | };
0101 |
0102 | // =====
0103 | // 中间件: GameController
0104 | // =====
0105 |
0106 | class GameController {
0107 |     Board _board;
0108 |     int _level = 0;
0109 |     int _step = 0;
0110 |     int _score = 0;
0111 |     bool _done = false;
0112 |     std::string _pending_line;
0113 |     std::vector<std::pair<int,int>> _last_path;
0114 |
0115 |     static int try_parse_level(const std::string& line, int& level, int& seed) {
0116 |         int lv, sd, N, steps;
0117 |         if (std::sscanf(line.c_str(), "LEVEL %d SEED %d SIZE %d STEPS %d", &lv, &sd, &N,
&steps) == 4) {
0118 |             level = lv;
0119 |             seed = sd;
0120 |             return N;
0121 |         }
0122 |         return 0;
0123 |     }
0124 |
0125 |     static bool try_parse_step(const std::string& line, int& step, int& score, bool&
valid) {

```

```

0126 |         char buf[16] = {};
0127 |         if (std::sscanf(line.c_str(), "STEP %d SCORE %d %15s", &step, &score, buf) >= 3)
    {
0128 |             valid = (std::string(buf) == "VALID");
0129 |             return true;
0130 |         }
0131 |         return false;
0132 |     }
0133 |
0134 |     static int gen_block(std::mt19937& rng, int level) {
0135 |         if (level <= 2) {
0136 |             return (rng() % 5) + 1;
0137 |         }
0138 |         if (level == 3) {
0139 |             return ((rng() % 100) < 15) ? 0 : (rng() % 5) + 1;
0140 |         }
0141 |         if (level == 4) {
0142 |             int color = (rng() % 5) + 1;
0143 |             return ((rng() % 100) < 10) ? -color : color;
0144 |         }
0145 |         if ((rng() % 100) < 15) return 0;
0146 |         int base = (rng() % 5) + 1;
0147 |         return ((rng() % 100) < 10) ? -base : base;
0148 |     }
0149 |
0150 |     static void init_queues(Board& b, int seed, int N, int level) {
0151 |         b.level = level;
0152 |         std::mt19937 rng(seed);
0153 |         b.drop_queue.assign(N, std::vector<int>(1000));
0154 |         b.queue_ptr.assign(N, 0);
0155 |         for (int c = 0; c < N; ++c) {
0156 |             for (int i = 0; i < 1000; ++i) {
0157 |                 b.drop_queue[c][i] = gen_block(rng, level);
0158 |             }
0159 |         }
0160 |     }
0161 |
0162 |     bool read_line(std::string& line) {
0163 |         if (!_pending_line.empty()) {
0164 |             line = std::move(_pending_line);
0165 |             _pending_line.clear();
0166 |             return true;
0167 |         }
0168 |         return static_cast<bool>(std::getline(std::cin, line));

```

```

0169 |     }
0170 |
0171 |     Board read_board(int N) {
0172 |         Board board(N);
0173 |         for (int row = 0; row < N; ++row) {
0174 |             std::string line;
0175 |             read_line(line);
0176 |             std::istringstream ls(line);
0177 |             for (int c = 0; c < N; ++c) ls >> board.at(row, c).value;
0178 |         }
0179 |         return board;
0180 |     }
0181 |
0182 |     void drain_trailing() {
0183 |         std::string line;
0184 |         while (std::cin.rdbuf()->in_avail() > 0) {
0185 |             if (!read_line(line)) break;
0186 |             if (line.empty() || line.find("LEVEL_END") != std::string::npos) continue;
0187 |             if (line.find("FINAL_SCORE") != std::string::npos) {
0188 |                 _done = true;
0189 |                 continue;
0190 |             }
0191 |             _pending_line = std::move(line);
0192 |             break;
0193 |         }
0194 |     }
0195 |
0196 | public:
0197 |     const Board& board() const { return _board; }
0198 |     int level() const { return _level; }
0199 |     int step() const { return _step; }
0200 |     int score() const { return _score; }
0201 |     bool done() const { return _done; }
0202 |
0203 |     bool update() {
0204 |         std::string first_line;
0205 |         while (true) {
0206 |             if (!read_line(first_line)) {
0207 |                 _done = true;
0208 |                 return false;
0209 |             }
0210 |             if (!first_line.empty()) break;
0211 |         }
0212 |

```



```

0213 |         if (first_line.find("LEVEL_END") != std::string::npos ||
0214 |             first_line.find("FINAL_SCORE") != std::string::npos) {
0215 |             _done = true;
0216 |             return false;
0217 |         }
0218 |
0219 |         int seed = 0;
0220 |         int new_N = try_parse_level(first_line, _level, seed);
0221 |         if (new_N > 0) {
0222 |             Board new_board = read_board(new_N);
0223 |             init_queues(new_board, seed, new_N, _level);
0224 |             _board = std::move(new_board);
0225 |             _step = 0;
0226 |             _score = 0;
0227 |             drain_trailing();
0228 |             return true;
0229 |         }
0230 |
0231 |         int step = 0;
0232 |         int score = 0;
0233 |         bool valid = false;
0234 |         if (try_parse_step(first_line, step, score, valid)) {
0235 |             _step = step;
0236 |             _score = score;
0237 |
0238 |             Board predicted = (valid && !_last_path.empty()) ?
_board.preview(_last_path) : _board;
0239 |
0240 |             Board new_board = read_board(_board.N);
0241 |             new_board.level = _level;
0242 |             new_board.drop_queue = std::move(predicted.drop_queue);
0243 |             new_board.queue_ptr = std::move(predicted.queue_ptr);
0244 |             _board = std::move(new_board);
0245 |             _last_path.clear();
0246 |
0247 |             drain_trailing();
0248 |             if (!_pending_line.empty()) {
0249 |                 int next_level = 0, next_seed = 0;
0250 |                 int next_N = try_parse_level(_pending_line, next_level, next_seed);
0251 |                 if (next_N > 0) {
0252 |                     _level = next_level;
0253 |                     _pending_line.clear();
0254 |                     Board nb = read_board(next_N);
0255 |                     init_queues(nb, next_seed, next_N, next_level);

```

```

0256 |             _board = std::move(nb);
0257 |             _step = 0;
0258 |             _score = 0;
0259 |             drain_trailing();
0260 |         }
0261 |     }
0262 |     return true;
0263 | }
0264 |
0265 |     _done = true;
0266 |     return false;
0267 | }
0268 |
0269 | void respond(const std::vector<std::pair<int,int>>& path) {
0270 |     _last_path = path;
0271 |     std::cout << path.size();
0272 |     for (auto [r, c] : path) std::cout << ' ' << r << ' ' << c;
0273 |     std::cout << '\n';
0274 |     std::cout.flush();
0275 | }
0276 | };
0277 |
0278 | // =====
0279 | // 工具函数
0280 | // =====
0281 |
0282 | constexpr int DR[] = {-1, 1, 0, 0};
0283 | constexpr int DC[] = { 0, 0,-1, 1};
0284 |
0285 | int path_score(int k) {
0286 |     double t = std::sqrt(static_cast<double>(k)) - 1.0;
0287 |     return 10 * k + 18 * static_cast<int>(t * t);
0288 | }
0289 |
0290 | int path_score(const Board& board, const std::vector<std::pair<int,int>>& path) {
0291 |     int s = path_score(static_cast<int>(path.size()));
0292 |     if (board.level < 4) return s;
0293 |
0294 |     std::vector<std::vector<bool>> in_path(board.N, std::vector<bool>(board.N, false));
0295 |     std::vector<std::vector<bool>> exploded(board.N, std::vector<bool>(board.N, false));
0296 |     for (auto [r, c] : path) in_path[r][c] = true;
0297 |
0298 |     for (auto [r, c] : path) {
0299 |         if (!board.at(r, c).is_bomb()) continue;

```

```

0300 |         for (int dr = -1; dr <= 1; ++dr) {
0301 |             for (int dc = -1; dc <= 1; ++dc) {
0302 |                 int nr = r + dr, nc = c + dc;
0303 |                 if (board.in_bounds(nr, nc) && !in_path[nr][nc] && !exploded[nr][nc]) {
0304 |                     exploded[nr][nc] = true;
0305 |                     s += 10;
0306 |                 }
0307 |             }
0308 |         }
0309 |     }
0310 |     return s;
0311 | }
0312 |
0313 | static int pure_color_structure_score(const Board& board) {
0314 |     if (board.level > 2) return 0;
0315 |
0316 |     int N = board.N;
0317 |     int total = N * N;
0318 |     std::vector<unsigned char> seen(total, 0);
0319 |
0320 |     int largest = 0;
0321 |     int isolated = 0;
0322 |     int same_edges = 0;
0323 |     int component_value = 0;
0324 |
0325 |     for (int r = 0; r < N; ++r) {
0326 |         for (int c = 0; c < N; ++c) {
0327 |             int color = board.at(r, c).color();
0328 |             if (c + 1 < N && board.at(r, c + 1).color() == color) ++same_edges;
0329 |             if (r + 1 < N && board.at(r + 1, c).color() == color) ++same_edges;
0330 |         }
0331 |     }
0332 |
0333 |     std::array<int, 144> q{};
0334 |     for (int idx = 0; idx < total; ++idx) {
0335 |         if (seen[idx]) continue;
0336 |
0337 |         int color = board.at(idx / N, idx % N).color();
0338 |         int head = 0;
0339 |         int tail = 0;
0340 |         int size = 0;
0341 |
0342 |         q[tail++] = idx;
0343 |         seen[idx] = 1;

```

```

0344 |
0345 |         while (head < tail) {
0346 |             int cur = q[head++];
0347 |             ++size;
0348 |
0349 |             int r = cur / N;
0350 |             int c = cur % N;
0351 |             for (int d = 0; d < 4; ++d) {
0352 |                 int nr = r + DR[d];
0353 |                 int nc = c + DC[d];
0354 |                 if (!board.in_bounds(nr, nc)) continue;
0355 |                 int ni = nr * N + nc;
0356 |                 if (seen[ni]) continue;
0357 |                 if (board.at(nr, nc).color() != color) continue;
0358 |                 seen[ni] = 1;
0359 |                 q[tail++] = ni;
0360 |             }
0361 |         }
0362 |
0363 |         largest = std::max(largest, size);
0364 |         if (size == 1) {
0365 |             ++isolated;
0366 |         } else {
0367 |             component_value += path_score(size);
0368 |         }
0369 |     }
0370 |
0371 |     return component_value + largest * 20 + same_edges * 8 - isolated * 6;
0372 | }
0373 |
0374 | // =====
0375 | // 更高上限的搜索策略
0376 | // =====
0377 |
0378 | struct MoveCandidate {
0379 |     std::vector<std::pair<int,int>> path;
0380 |     int score = 0;
0381 |     int heuristic_score = 0;
0382 |     int length = 0;
0383 |     int bomb_count = 0;
0384 |     int blast_cells = 0;
0385 | };
0386 |
0387 | static bool better_candidate(const MoveCandidate& a, const MoveCandidate& b) {

```

```

0388 |         if (a.heuristic_score != b.heuristic_score) return a.heuristic_score >
b.heuristic_score;
0389 |     if (a.score != b.score) return a.score > b.score;
0390 |     if (a.length != b.length) return a.length > b.length;
0391 |     if (a.blast_cells != b.blast_cells) return a.blast_cells > b.blast_cells;
0392 |     return a.bomb_count > b.bomb_count;
0393 | }
0394 |
0395 | static uint64_t hash_path_indices(const std::vector<int>& path) {
0396 |     uint64_t h = 1469598103934665603ULL;
0397 |     for (int x : path) {
0398 |         h ^= static_cast<uint64_t>(x + 1);
0399 |         h *= 1099511628211ULL;
0400 |     }
0401 |     h ^= static_cast<uint64_t>(path.size()) << 32;
0402 |     return h;
0403 | }
0404 |
0405 | static uint64_t hash_path_pairs(const std::vector<std::pair<int,int>>& path, int N) {
0406 |     uint64_t h = 1469598103934665603ULL;
0407 |     for (auto [r, c] : path) {
0408 |         int idx = r * N + c;
0409 |         h ^= static_cast<uint64_t>(idx + 1);
0410 |         h *= 1099511628211ULL;
0411 |     }
0412 |     h ^= static_cast<uint64_t>(path.size()) << 32;
0413 |     return h;
0414 | }
0415 |
0416 | class CandidatePool {
0417 |     int _limit = 0;
0418 |     std::vector<MoveCandidate> _items;
0419 |     std::unordered_set<uint64_t> _seen;
0420 |
0421 |     static bool better_tuple(int heuristic_a, int score_a, int len_a, int blast_a, int
bombs_a,
0422 |                             int heuristic_b, int score_b, int len_b, int blast_b, int
bombs_b) {
0423 |         if (heuristic_a != heuristic_b) return heuristic_a > heuristic_b;
0424 |         if (score_a != score_b) return score_a > score_b;
0425 |         if (len_a != len_b) return len_a > len_b;
0426 |         if (blast_a != blast_b) return blast_a > blast_b;
0427 |         return bombs_a > bombs_b;
0428 |     }

```

```

0429 |
0430 |     void sort_trim() {
0431 |         std::sort(_items.begin(), _items.end(), better_candidate);
0432 |         if (static_cast<int>(_items.size()) > _limit) {
0433 |             _items.resize(_limit);
0434 |         }
0435 |     }
0436 |
0437 | public:
0438 |     explicit CandidatePool(int limit = 0) : _limit(limit) {}
0439 |
0440 |     bool full() const {
0441 |         return _limit > 0 && static_cast<int>(_items.size()) >= _limit;
0442 |     }
0443 |
0444 |     int worst_score() const {
0445 |         if (_items.empty()) return std::numeric_limits<int>::min();
0446 |         return _items.back().score;
0447 |     }
0448 |
0449 |     bool worth_consider(int heuristic_score, int score, int length, int blast_cells, int
bomb_count) const {
0450 |         if (_limit <= 0) return false;
0451 |         if (static_cast<int>(_items.size()) < _limit) return true;
0452 |         const MoveCandidate& worst = _items.back();
0453 |         return better_tuple(heuristic_score, score, length, blast_cells, bomb_count,
0454 |                             worst.heuristic_score, worst.score, worst.length,
worst.blast_cells, worst.bomb_count);
0455 |     }
0456 |
0457 |     void add_from_indices(const std::vector<int>& idx_path, int N, int score,
0458 |                         int heuristic_score, int bomb_count, int blast_cells) {
0459 |         int length = static_cast<int>(idx_path.size());
0460 |         if (!worth_consider(heuristic_score, score, length, blast_cells, bomb_count))
return;
0461 |         uint64_t h = hash_path_indices(idx_path);
0462 |         if (!_seen.insert(h).second) return;
0463 |
0464 |         MoveCandidate cand;
0465 |         cand.score = score;
0466 |         cand.heuristic_score = heuristic_score;
0467 |         cand.length = length;
0468 |         cand.bomb_count = bomb_count;
0469 |         cand.blast_cells = blast_cells;

```

```

0470 |         cand.path.reserve(idx_path.size());
0471 |         for (int idx : idx_path) {
0472 |             cand.path.push_back({idx / N, idx % N});
0473 |         }
0474 |         _items.push_back(std::move(cand));
0475 |         sort_trim();
0476 |     }
0477 |
0478 |     void add_candidate(const MoveCandidate& cand, int N) {
0479 |         if (!worth_consider(cand.heuristic_score, cand.score, cand.length,
cand.blast_cells, cand.bomb_count)) return;
0480 |         uint64_t h = hash_path_pairs(cand.path, N);
0481 |         if (!_seen.insert(h).second) return;
0482 |         _items.push_back(cand);
0483 |         sort_trim();
0484 |     }
0485 |
0486 |     const std::vector<MoveCandidate>& items() const { return _items; }
0487 | };
0488 |
0489 | struct ComponentInfo {
0490 |     std::vector<int> cells;
0491 |     int anchor = 1;
0492 |     int bomb_count = 0;
0493 |     int wildcard_count = 0;
0494 |     bool has_colored = false;
0495 |     int optimistic_score = 0;
0496 | };
0497 |
0498 | struct CandidateGenConfig {
0499 |     int total_budget = 0;
0500 |     int component_budget_base = 0;
0501 |     int component_budget_per_cell = 0;
0502 |     int candidate_limit = 0;
0503 |     int per_component_limit = 0;
0504 |     int start_limit = 0;
0505 | };
0506 |
0507 | struct SearchStage {
0508 |     CandidateGenConfig gen;
0509 |     int branch_width = 1;
0510 |     int deadlock_penalty = 0;
0511 |     double future_weight = 0.0;
0512 |     double immediate_weight = 1.0;

```

```

0513 |     double shape_weight = 0.0;
0514 | };
0515 |
0516 | struct ComponentSearchResult {
0517 |     std::vector<MoveCandidate> candidates;
0518 |     int nodes_used = 0;
0519 | };
0520 |
0521 | static bool matches_anchor(const Cell& cell, int anchor) {
0522 |     return cell.color() == 0 || cell.color() == anchor;
0523 | }
0524 |
0525 | static std::vector<ComponentInfo> collect_components(const Board& board) {
0526 |     int N = board.N;
0527 |     int total = N * N;
0528 |     std::vector<ComponentInfo> comps;
0529 |     std::vector<unsigned char> wildcard_only_seen(total, 0);
0530 |
0531 |     for (int anchor = 1; anchor <= 5; ++anchor) {
0532 |         std::vector<unsigned char> seen(total, 0);
0533 |         for (int idx = 0; idx < total; ++idx) {
0534 |             if (seen[idx]) continue;
0535 |             int r = idx / N;
0536 |             int c = idx % N;
0537 |             if (!matches_anchor(board.at(r, c), anchor)) continue;
0538 |
0539 |             std::queue<int> q;
0540 |             q.push(idx);
0541 |             seen[idx] = 1;
0542 |
0543 |             ComponentInfo comp;
0544 |             comp.anchor = anchor;
0545 |
0546 |             while (!q.empty()) {
0547 |                 int cur = q.front();
0548 |                 q.pop();
0549 |
0550 |                 int cr = cur / N;
0551 |                 int cc = cur % N;
0552 |                 const Cell& cell = board.at(cr, cc);
0553 |
0554 |                 comp.cells.push_back(cur);
0555 |                 if (cell.color() != 0) comp.has_colored = true;
0556 |                 if (cell.is_bomb()) ++comp.bomb_count;

```


[illegible]

```

0599 |     ComponentSearchResult result;
0600 |     if (node_budget <= 0 || local_limit <= 0) return result;
0601 |
0602 |     int N = board.N;
0603 |     int total = N * N;
0604 |     bool bomb_level = (board.level >= 4);
0605 |
0606 |     std::vector<unsigned char> in_comp(total, 0);
0607 |     for (int idx : comp.cells) in_comp[idx] = 1;
0608 |
0609 |     std::vector<int> degree(total, 0);
0610 |     for (int idx : comp.cells) {
0611 |         int r = idx / N;
0612 |         int c = idx % N;
0613 |         int deg = 0;
0614 |         for (int d = 0; d < 4; ++d) {
0615 |             int nr = r + DR[d], nc = c + DC[d];
0616 |             if (!board.in_bounds(nr, nc)) continue;
0617 |             int ni = nr * N + nc;
0618 |             if (in_comp[ni]) ++deg;
0619 |         }
0620 |         degree[idx] = deg;
0621 |     }
0622 |
0623 |     bool pure_color_aggressive = (board.level <= 2);
0624 |     bool pure_color_connector_mode = pure_color_aggressive &&
0625 |         static_cast<int>(comp.cells.size()) >= 8;
0626 |     bool wildcard_bridge_level = (board.level == 3);
0627 |
0628 |     std::vector<int> starts = comp.cells;
0629 |     std::sort(starts.begin(), starts.end(), [&](int a, int b) {
0630 |         const Cell& ca = board.at(a / N, a % N);
0631 |         const Cell& cb = board.at(b / N, b % N);
0632 |         if (pure_color_aggressive) {
0633 |             if (pure_color_connector_mode) {
0634 |                 if (degree[a] != degree[b]) return degree[a] > degree[b];
0635 |                 int ea = std::min({a / N, a % N, N - 1 - a / N, N - 1 - a % N});
0636 |                 int eb = std::min({b / N, b % N, N - 1 - b / N, N - 1 - b % N});
0637 |                 return ea > eb;
0638 |             }
0639 |             if (degree[a] != degree[b]) return degree[a] < degree[b];
0640 |             int ea = std::min({a / N, a % N, N - 1 - a / N, N - 1 - a % N});
0641 |             int eb = std::min({b / N, b % N, N - 1 - b / N, N - 1 - b % N});
0642 |             return ea < eb;

```

```

0643 |     }
0644 |     if (wildcard_bridge_level) {
0645 |         auto wildcard_rank = [&](int idx, const Cell& cell) {
0646 |             if (!cell.is_wildcard()) return 0;
0647 |             if (degree[idx] >= 3) return 3;
0648 |             if (degree[idx] == 2) return 2;
0649 |             return 1;
0650 |         };
0651 |         int ra = wildcard_rank(a, ca);
0652 |         int rb = wildcard_rank(b, cb);
0653 |         if (ra != rb) return ra < rb;
0654 |     }
0655 |     if (degree[a] != degree[b]) return degree[a] < degree[b];
0656 |     if (ca.is_bomb() != cb.is_bomb()) return ca.is_bomb() && !cb.is_bomb();
0657 |     if (ca.is_wildcard() != cb.is_wildcard()) return ca.is_wildcard()
&& !cb.is_wildcard();
0658 |     int ea = std::min({a / N, a % N, N - 1 - a / N, N - 1 - a % N});
0659 |     int eb = std::min({b / N, b % N, N - 1 - b / N, N - 1 - b % N});
0660 |     return ea < eb;
0661 | });
0662 | if (static_cast<int>(starts.size()) > start_limit) starts.resize(start_limit);
0663 |
0664 | CandidatePool pool(local_limit);
0665 | std::vector<unsigned char> visited(total, 0);
0666 | std::vector<int> cover(total, 0);
0667 | std::vector<int> bfs_mark(total, 0);
0668 | std::vector<int> path;
0669 | path.reserve(comp.cells.size());
0670 |
0671 | int nodes_used = 0;
0672 | int bonus_cells = 0;
0673 | int bombs_in_path = 0;
0674 | int wildcard_bridge_cost = 0;
0675 | int bfs_token = 0;
0676 |
0677 | auto chain_bias = [&](int len) {
0678 |     if (!pure_color_aggressive) return 0;
0679 |     int bias = pure_color_connector_mode ? len * 3 : len * 6;
0680 |     if (len >= 8) bias += pure_color_connector_mode ? 8 : 18;
0681 |     if (len >= 10) bias += pure_color_connector_mode ? 10 : 20;
0682 |     if (len <= 3) bias -= pure_color_connector_mode ? 8 : 16;
0683 |     return bias;
0684 | };
0685 |

```

```

0686 |     auto wildcard_cost = [&](int idx) {
0687 |         if (!wildcard_bridge_level) return 0;
0688 |         const Cell& cell = board.at(idx / N, idx % N);
0689 |         if (!cell.is_wildcard()) return 0;
0690 |         if (degree[idx] >= 4) return 24;
0691 |         if (degree[idx] == 3) return 16;
0692 |         if (degree[idx] == 2) return 6;
0693 |         return -4;
0694 |     };
0695 |
0696 |     auto local_future_potential = [&](int end_idx, int len) {
0697 |         int r = end_idx / N;
0698 |         int c = end_idx % N;
0699 |         int exits = 0;
0700 |         int branch_mass = 0;
0701 |         for (int d = 0; d < 4; ++d) {
0702 |             int nr = r + DR[d];
0703 |             int nc = c + DC[d];
0704 |             if (!board.in_bounds(nr, nc)) continue;
0705 |             int ni = nr * N + nc;
0706 |             if (!in_comp[ni] || visited[ni]) continue;
0707 |             ++exits;
0708 |
0709 |             int next_deg = 0;
0710 |             for (int dd = 0; dd < 4; ++dd) {
0711 |                 int nnr = nr + DR[dd];
0712 |                 int nnc = nc + DC[dd];
0713 |                 if (!board.in_bounds(nnr, nnc)) continue;
0714 |                 int nni = nnr * N + nnc;
0715 |                 if (in_comp[nni] && !visited[nni]) ++next_deg;
0716 |             }
0717 |             branch_mass += next_deg;
0718 |         }
0719 |
0720 |         int edge_depth = std::min({r, c, N - 1 - r, N - 1 - c});
0721 |         int remaining = static_cast<int>(comp.cells.size()) - len;
0722 |
0723 |         if (pure_color_aggressive) {
0724 |             int potential = exits * 24 + branch_mass * 4 + std::min(remaining, 8) * 3;
0725 |             if (pure_color_connector_mode) potential += edge_depth * 10;
0726 |             return potential;
0727 |         }
0728 |         if (wildcard_bridge_level) {
0729 |             return exits * 18 + branch_mass * 3;

```

```

0730 |     }
0731 |     return 0;
0732 | };
0733 |
0734 | auto push_cell = [&](int idx) {
0735 |     if (cover[idx] > 0) --bonus_cells;
0736 |     visited[idx] = 1;
0737 |     path.push_back(idx);
0738 |     wildcard_bridge_cost += wildcard_cost(idx);
0739 |
0740 |     if (!bomb_level) return;
0741 |
0742 |     int r = idx / N;
0743 |     int c = idx % N;
0744 |     if (!board.at(r, c).is_bomb()) return;
0745 |
0746 |     ++bombs_in_path;
0747 |     for (int dr = -1; dr <= 1; ++dr) {
0748 |         for (int dc = -1; dc <= 1; ++dc) {
0749 |             int nr = r + dr, nc = c + dc;
0750 |             if (!board.in_bounds(nr, nc)) continue;
0751 |             int ni = nr * N + nc;
0752 |             if (!visited[ni] && cover[ni] == 0) ++bonus_cells;
0753 |             ++cover[ni];
0754 |         }
0755 |     }
0756 | };
0757 |
0758 | auto pop_cell = [&](int idx) {
0759 |     if (bomb_level) {
0760 |         int r = idx / N;
0761 |         int c = idx % N;
0762 |         if (board.at(r, c).is_bomb()) {
0763 |             for (int dr = -1; dr <= 1; ++dr) {
0764 |                 for (int dc = -1; dc <= 1; ++dc) {
0765 |                     int nr = r + dr, nc = c + dc;
0766 |                     if (!board.in_bounds(nr, nc)) continue;
0767 |                     int ni = nr * N + nc;
0768 |                     if (!visited[ni] && cover[ni] == 1) --bonus_cells;
0769 |                     --cover[ni];
0770 |                 }
0771 |             }
0772 |             --bombs_in_path;
0773 |         }

```

```

0774 |     }
0775 |
0776 |     wildcard_bridge_cost -= wildcard_cost(idx);
0777 |     visited[idx] = 0;
0778 |     if (cover[idx] > 0) ++bonus_cells;
0779 |     path.pop_back();
0780 | };
0781 |
0782 | auto reachable_from_end = [&](int start_idx) {
0783 |     ++bfs_token;
0784 |     if (bfs_token == std::numeric_limits<int>::max()) {
0785 |         std::fill(bfs_mark.begin(), bfs_mark.end(), 0);
0786 |         bfs_token = 1;
0787 |     }
0788 |
0789 |     std::array<int, 144> q{};
0790 |     int head = 0, tail = 0;
0791 |     q[tail++] = start_idx;
0792 |     bfs_mark[start_idx] = bfs_token;
0793 |     int count = 0;
0794 |
0795 |     while (head < tail) {
0796 |         int cur = q[head++];
0797 |         ++count;
0798 |         int r = cur / N;
0799 |         int c = cur % N;
0800 |         for (int d = 0; d < 4; ++d) {
0801 |             int nr = r + DR[d], nc = c + DC[d];
0802 |             if (!board.in_bounds(nr, nc)) continue;
0803 |             int ni = nr * N + nc;
0804 |             if (!in_comp[ni]) continue;
0805 |             if (visited[ni] && ni != start_idx) continue;
0806 |             if (bfs_mark[ni] == bfs_token) continue;
0807 |             bfs_mark[ni] = bfs_token;
0808 |             q[tail++] = ni;
0809 |         }
0810 |     }
0811 |     return count;
0812 | };
0813 |
0814 | auto dfs = [&](auto& self, int cur_idx) -> void {
0815 |     if (nodes_used >= node_budget) return;
0816 |     ++nodes_used;
0817 |

```

```

0818 |         int len = static_cast<int>(path.size());
0819 |         if (len >= 2) {
0820 |             int score = path_score(len) + 10 * bonus_cells;
0821 |             int heuristic_score = score + chain_bias(len) +
local_future_potential(cur_idx, len) - wildcard_bridge_cost;
0822 |             pool.add_from_indices(path, N, score, heuristic_score, bombs_in_path,
bonus_cells);
0823 |         }
0824 |
0825 |         if (nodes_used >= node_budget || len == static_cast<int>(comp.cells.size()))
return;
0826 |
0827 |         if (pool.full()) {
0828 |             int reachable = reachable_from_end(cur_idx);
0829 |             int optimistic_len = len + reachable - 1;
0830 |             int optimistic_bonus = bonus_cells;
0831 |             if (bomb_level) {
0832 |                 int remain_bombs = comp.bomb_count - bombs_in_path;
0833 |                 optimistic_bonus = std::min(board.N * board.N - 2, bonus_cells + 8 *
remain_bombs);
0834 |             }
0835 |             int optimistic_score = path_score(optimistic_len) + 10 * optimistic_bonus;
0836 |             if (optimistic_score <= pool.worst_score()) return;
0837 |         }
0838 |
0839 |         struct NextStep {
0840 |             int priority;
0841 |             int rem_deg;
0842 |             int idx;
0843 |         };
0844 |         std::array<NextStep, 4> nexts{};
0845 |         int next_count = 0;
0846 |
0847 |         int r = cur_idx / N;
0848 |         int c = cur_idx % N;
0849 |         for (int d = 0; d < 4; ++d) {
0850 |             int nr = r + DR[d], nc = c + DC[d];
0851 |             if (!board.in_bounds(nr, nc)) continue;
0852 |             int ni = nr * N + nc;
0853 |             if (!in_comp[ni] || visited[ni]) continue;
0854 |
0855 |             int rem_deg = 0;
0856 |             for (int dd = 0; dd < 4; ++dd) {
0857 |                 int nnr = nr + DR[dd], nnc = nc + DC[dd];

```

```

0858 |         if (!board.in_bounds(nnr, nnc)) continue;
0859 |         int nni = nnr * N + nnc;
0860 |         if (in_comp[nni] && !visited[nni]) ++rem_deg;
0861 |     }
0862 |
0863 |     int edge_depth = std::min({nr, nc, N - 1 - nr, N - 1 - nc});
0864 |     int priority = rem_deg * 100;
0865 |     if (pure_color_connector_mode) {
0866 |         priority = -rem_deg * 100 - edge_depth * 12;
0867 |     }
0868 |     if (bomb_level && board.at(nr, nc).is_bomb()) priority -= 35;
0869 |     if (wildcard_bridge_level && board.at(nr, nc).is_wildcard()) {
0870 |         if (degree[ni] >= 3) priority += 36;
0871 |         else if (degree[ni] == 2) priority += 10;
0872 |         else priority -= 8;
0873 |     } else if (board.at(nr, nc).is_wildcard()) {
0874 |         priority -= 12;
0875 |     }
0876 |     if (!pure_color_connector_mode) priority += edge_depth;
0877 |     nexts[next_count++] = {priority, rem_deg, ni};
0878 | }
0879 |
0880 | for (int i = 1; i < next_count; ++i) {
0881 |     NextStep key = nexts[i];
0882 |     int j = i - 1;
0883 |     while (j >= 0) {
0884 |         bool should_shift = false;
0885 |         if (nexts[j].priority != key.priority) {
0886 |             should_shift = nexts[j].priority > key.priority;
0887 |         } else {
0888 |             should_shift = nexts[j].rem_deg > key.rem_deg;
0889 |         }
0890 |         if (!should_shift) break;
0891 |         nexts[j + 1] = nexts[j];
0892 |         --j;
0893 |     }
0894 |     nexts[j + 1] = key;
0895 | }
0896 |
0897 | for (int i = 0; i < next_count; ++i) {
0898 |     push_cell(nexts[i].idx);
0899 |     self(self, nexts[i].idx);
0900 |     pop_cell(nexts[i].idx);
0901 |     if (nodes_used >= node_budget) return;

```



```

0902 |     }
0903 | };
0904 |
0905 |     for (int start_idx : starts) {
0906 |         if (nodes_used >= node_budget) break;
0907 |         if (pool.full() && comp.optimistic_score <= pool.worst_score()) break;
0908 |         push_cell(start_idx);
0909 |         dfs(dfs, start_idx);
0910 |         pop_cell(start_idx);
0911 |     }
0912 |
0913 |     result.candidates = pool.items();
0914 |     result.nodes_used = nodes_used;
0915 |     return result;
0916 | }
0917 |
0918 | static std::vector<MoveCandidate> generate_candidates(const Board& board, const
CandidateGenConfig& cfg) {
0919 |     std::vector<MoveCandidate> empty;
0920 |     if (cfg.total_budget <= 0 || cfg.candidate_limit <= 0) return empty;
0921 |
0922 |     std::vector<ComponentInfo> comps = collect_components(board);
0923 |     if (comps.empty()) return empty;
0924 |
0925 |     CandidatePool global_pool(cfg.candidate_limit);
0926 |     int remaining_budget = cfg.total_budget;
0927 |
0928 |     for (const ComponentInfo& comp : comps) {
0929 |         if (remaining_budget <= 0) break;
0930 |         if (global_pool.full() && comp.optimistic_score <= global_pool.worst_score())
continue;
0931 |
0932 |         int component_budget = cfg.component_budget_base +
0933 |                                 cfg.component_budget_per_cell *
static_cast<int>(comp.cells.size());
0934 |         component_budget = std::min(component_budget, remaining_budget);
0935 |         if (component_budget <= 0) break;
0936 |
0937 |         ComponentSearchResult local = search_component(
0938 |             board,
0939 |             comp,
0940 |             component_budget,
0941 |             std::max(1, std::min(cfg.start_limit, static_cast<int>(comp.cells.size()))),
0942 |             cfg.per_component_limit

```

```

0943 |         );
0944 |
0945 |         remaining_budget -= local.nodes_used;
0946 |         for (const MoveCandidate& cand : local.candidates) {
0947 |             global_pool.add_candidate(cand, board.N);
0948 |         }
0949 |     }
0950 |
0951 |     return global_pool.items();
0952 | }
0953 |
0954 | static int estimate_future_score(const Board& board,
0955 |                                 const std::vector<SearchStage>& stages,
0956 |                                 int stage_idx) {
0957 |     if (stage_idx >= static_cast<int>(stages.size()) || board.is_deadlocked()) return 0;
0958 |
0959 |     const SearchStage& stage = stages[stage_idx];
0960 |     std::vector<MoveCandidate> cands = generate_candidates(board, stage.gen);
0961 |     if (cands.empty()) return 0;
0962 |
0963 |     int branch = std::min(stage.branch_width, static_cast<int>(cands.size()));
0964 |     int best = 0;
0965 |
0966 |     for (int i = 0; i < branch; ++i) {
0967 |         const MoveCandidate& cand = cands[i];
0968 |         Board next_board = board.preview(cand.path);
0969 |         int future = estimate_future_score(next_board, stages, stage_idx + 1);
0970 |         int shape = pure_color_structure_score(next_board);
0971 |
0972 |         int total = static_cast<int>(std::lround(stage.immediate_weight * cand.score));
0973 |         if (stage_idx + 1 < static_cast<int>(stages.size())) {
0974 |             total += static_cast<int>(std::lround(stage.future_weight * future));
0975 |         }
0976 |         total += static_cast<int>(std::lround(stage.shape_weight * shape));
0977 |         if (next_board.is_deadlocked()) total -= stage.deadlock_penalty;
0978 |         best = std::max(best, total);
0979 |     }
0980 |
0981 |     return best;
0982 | }
0983 |
0984 | static std::vector<SearchStage> make_search_plan(const Board& board, int step) {
0985 |     if (board.level == 1) {
0986 |         return {

```

```

0987 |             step < 14 ? SearchStage{{360000, 7200, 3300, 38, 8, 16}, 22, 110, 1.02,
0.86, 0.18}
0988 |                                     : step < 32 ? SearchStage{{330000, 6800, 3200, 36, 8, 15}, 21,
100, 0.94, 0.96, 0.10}
0989 |                                     : SearchStage{{280000, 6000, 2900, 34, 8, 14}, 18,
90, 0.80, 1.06, 0.02},
0990 |             step < 14 ? SearchStage{{110000, 2800, 1400, 14, 4, 10}, 7, 50, 0.78,
0.92, 0.08}
0991 |                                     : step < 32 ? SearchStage{{ 98000, 2500, 1280, 13, 4, 10}, 6,
45, 0.66, 1.00, 0.04}
0992 |                                     : SearchStage{{ 80000, 2200, 1150, 12, 4, 9}, 5,
40, 0.54, 1.08, 0.00},
0993 |             {{24000, 950, 520, 6, 2, 7}, 1, 0, 0.00, 1.00, 0.00},
0994 |             };
0995 |     }
0996 |
0997 |     if (board.level == 2) {
0998 |         return {
0999 |             step < 10 ? SearchStage{{340000, 7000, 3250, 36, 8, 16}, 22, 100, 0.98,
0.92, 0.14}
1000 |                                     : step < 28 ? SearchStage{{320000, 6500, 3050, 35, 8, 15}, 20,
95, 0.90, 1.00, 0.08}
1001 |                                     : SearchStage{{290000, 6000, 2850, 34, 8, 14}, 18,
90, 0.76, 1.10, 0.00},
1002 |             step < 10 ? SearchStage{{105000, 2700, 1360, 14, 4, 10}, 7, 50, 0.74,
0.96, 0.06}
1003 |                                     : step < 28 ? SearchStage{{ 95000, 2450, 1240, 13, 4, 10}, 6,
45, 0.62, 1.02, 0.03}
1004 |                                     : SearchStage{{ 82000, 2200, 1140, 12, 4, 9}, 5,
40, 0.50, 1.08, 0.00},
1005 |             {{24000, 950, 520, 6, 2, 7}, 1, 0, 0.00, 1.00, 0.00},
1006 |             };
1007 |     }
1008 |
1009 |     if (board.level == 3) {
1010 |         return {
1011 |             step < 18 ? SearchStage{{240000, 5300, 2550, 28, 7, 13}, 14, 70, 0.62, 1.08,
0.00}
1012 |                                     : SearchStage{{225000, 5000, 2400, 27, 7, 13}, 12, 70, 0.56, 1.14,
0.00},
1013 |             step < 18 ? SearchStage{{ 72000, 1950, 1080, 11, 4, 8}, 4, 35, 0.40, 1.02,
0.00}
1014 |                                     : SearchStage{{ 68000, 1850, 1020, 10, 4, 8}, 4, 35, 0.34, 1.05,
0.00},

```

```

1015 |         {{18000, 700, 400, 5, 2, 6}, 1, 0, 0.00, 1.00, 0.00},
1016 |     };
1017 | }
1018 |
1019 | if (board.level == 4) {
1020 |     return {
1021 |         {{190000, 4200, 2100, 24, 6, 12}, 10, 70, 0.78, 1.00, 0.00},
1022 |         {{ 60000, 1800,  950, 10, 4,  8},  4, 35, 0.58, 1.00, 0.00},
1023 |         {{ 15000,  600,  350,  5, 2,  6},  1,  0, 0.00, 1.00, 0.00},
1024 |     };
1025 | }
1026 |
1027 | if (board.level == 5) {
1028 |     return {
1029 |         {{150000, 4000, 1700, 18, 5, 10}, 8, 70, 0.75, 1.00, 0.00},
1030 |         {{ 35000, 1200,  700,  8, 3,  7}, 3, 35, 0.55, 1.00, 0.00},
1031 |         {{ 10000,  450,  250,  4, 2,  5}, 1,  0, 0.00, 1.00, 0.00},
1032 |     };
1033 | }
1034 |
1035 | return {
1036 |     {{190000, 4200, 2100, 24, 6, 12}, 10, 70, 0.78, 1.00, 0.00},
1037 |     {{ 60000, 1800,  950, 10, 4,  8},  4, 35, 0.58, 1.00, 0.00},
1038 |     {{ 15000,  600,  350,  5, 2,  6},  1,  0, 0.00, 1.00, 0.00},
1039 | };
1040 | }
1041 |
1042 | static std::vector<std::pair<int,int>> fallback_path(const Board& board) {
1043 |     for (int r = 0; r < board.N; ++r) {
1044 |         for (int c = 0; c < board.N; ++c) {
1045 |             int color1 = board.at(r, c).color();
1046 |             if (c + 1 < board.N) {
1047 |                 int color2 = board.at(r, c + 1).color();
1048 |                 if (color1 == color2 || color1 == 0 || color2 == 0) {
1049 |                     return {{r, c}, {r, c + 1}};
1050 |                 }
1051 |             }
1052 |             if (r + 1 < board.N) {
1053 |                 int color2 = board.at(r + 1, c).color();
1054 |                 if (color1 == color2 || color1 == 0 || color2 == 0) {
1055 |                     return {{r, c}, {r + 1, c}};
1056 |                 }
1057 |             }
1058 |         }

```

```

1059 |     }
1060 |     return {{0, 0}, {0, 1}};
1061 | }
1062 |
1063 | std::vector<std::pair<int,int>> find_best_path(const Board& board, int step) {
1064 |     std::vector<SearchStage> plan = make_search_plan(board, step);
1065 |     std::vector<MoveCandidate> roots = generate_candidates(board, plan.front().gen);
1066 |     if (roots.empty()) return fallback_path(board);
1067 |
1068 |     int branch = std::min(plan.front().branch_width, static_cast<int>(roots.size()));
1069 |     int best_total = std::numeric_limits<int>::min();
1070 |     std::vector<std::pair<int,int>> best_path = roots.front().path;
1071 |
1072 |     for (int i = 0; i < branch; ++i) {
1073 |         const MoveCandidate& cand = roots[i];
1074 |         Board next_board = board.preview(cand.path);
1075 |         int future = estimate_future_score(next_board, plan, 1);
1076 |         int shape = pure_color_structure_score(next_board);
1077 |
1078 |         int total = static_cast<int>(std::lround(plan.front().immediate_weight *
1079 | cand.score));
1080 |         if (plan.size() > 1) {
1081 |             total += static_cast<int>(std::lround(plan.front().future_weight * future));
1082 |         }
1083 |         total += static_cast<int>(std::lround(plan.front().shape_weight * shape));
1084 |         if (next_board.is_deadlocked()) total -= plan.front().deadlock_penalty;
1085 |
1086 |         if (total > best_total) {
1087 |             best_total = total;
1088 |             best_path = cand.path;
1089 |         }
1090 |     }
1091 |     return best_path;
1092 | }
1093 |
1094 | int main() {
1095 |     std::ios::sync_with_stdio(false);
1096 |     std::cin.tie(nullptr);
1097 |
1098 |     GameController ctl;
1099 |     while (ctl.update()) {
1100 |         auto path = find_best_path(ctl.board(), ctl.step());
1101 |         ctl.respond(path);

```

```
1102 |     }  
1103 |     return 0;  
1104 | }
```